A PLAIN-LANGUAGE INTRODUCTION

OpenSpec: agree on the change before you build it

How it works, and why it makes delivery safer

Paul Rule · November 2026

WHY THIS MATTERS

Faster coding makes unclear intent more expensive

SITUATION

We build more, faster, and AI assistants now write a growing share of the code.

COMPLICATION

Intent still lives in tickets, chats and people's heads. Reviewers see a diff, not the agreement behind it, so misunderstandings surface late.

QUESTION

How do we agree on what to build, and keep that agreement, without slowing down?

THE ANSWER

OpenSpec on one page

OpenSpec puts a short, agreed spec in front of every change, so teams build what they agreed and keep a living record of how the system behaves.

1 · HOW IT WORKS

It is simple

Two folders and a three-step loop: propose, apply, archive.

2 · WHY IT IS SAFER

It reduces risk

The team agrees on behaviour in plain language before code is written.

3 · WHY IT PAYS OFF

It compounds

Specs stay current and live in Git, readable by people and AI.

Installing takes two commands and adds only plain files to your repo

1 Install the command-line tool, once

```
npm install -g @fission-ai/openspec@latest
```

Needs Node.js 20.19 or later.

2 Set it up in each repository

```
cd your-project && openspec init
```

Asks which AI tools you use: Claude Code, Cursor, GitHub Copilot, Codex and many more.

What gets added

`openspec/specs/`	Empty, fills as you archive
`openspec/changes/`	Work in progress
`openspec/config.yaml`	Context and rules
`.claude/commands/opsx/`	Slash commands
`.claude/skills/openspec-*/`	Skills for the assistant

AI paths shown for Claude Code; each tool gets its own. No server, nothing runs in the background.

Two folders hold the work: what is true today, and what we propose to change

`openspec/specs/`

What is true today

- One folder per capability, such as auth or billing
- Each spec lists requirements and scenarios
- Changes only when a change is archived

`openspec/changes/add-2fa/`

What we propose to change

`proposal.md`	Why and what
`design.md`	How, and the key decisions
`tasks.md`	The checklist to build it
`specs/`	The delta: what behaviour changes

Every piece of OpenSpec in one tree

```
your-project/
├── openspec/
│   ├── config.yaml
│   ├── specs/
│   │   └── auth/
│   │       └── spec.md
│   └── changes/
│       ├── add-2fa/
│       │   ├── proposal.md
│       │   ├── design.md
│       │   ├── tasks.md
│       │   └── specs/auth/spec.md
│       └── archive/
│           └── 2026-10-02-add-login/
├── .claude/
│   ├── commands/opsx/
│   └── skills/openspec-*/
└── src/
```

`config.yaml`

Project context and team rules for the AI.

`specs/`

What is true today, one folder per capability.

`changes/add-2fa/`

One folder per proposed change, with its delta.

`changes/archive/`

Finished changes, dated, with their reasoning.

`.claude/`

Slash commands and skills for your AI tool.

One config file tells the AI how your project works

openspec/config.yaml

```yaml
schema: spec-driven
context: |
  Product: an online bookshop.
  Money is stored in cents.

rules:
  proposal:
    - Name the user who benefits
  tasks:
    - Every task includes its tests
```

schema

Which artifacts a change has, and in what order.

context

Facts about the product and stack, given to the AI on every change.

rules

The team’s standards for each artifact: proposal, design, specs, tasks.

Every change follows the same three steps: propose, apply, archive

1

Propose

Describe the change. The AI assistant drafts the proposal, design, tasks and spec delta. The team reviews and agrees.

`/opsx:propose`

2

Apply

Build the tasks against the agreed delta, ticking each one off as it is done.

`/opsx:apply`

3

Archive

Merge the delta into the main specs. The change folder moves to the archive, reasoning intact.

`/opsx:archive`

1 · HOW IT WORKS

A spec delta states behaviour in plain, testable language

```
## ADDED Requirements

### Requirement: Reset links expire
The system SHALL reject a password
reset link older than 30 minutes.

#### Scenario: Expired link
- **WHEN** a user opens a 31-minute-old link
- **THEN** they see “Link expired”
- **AND** they can request a new link
```

ADDED · MODIFIED · REMOVED

Says how behaviour changes, not how the code changes.

SHALL

One clear, checkable rule per requirement.

WHEN · THEN

A concrete example that becomes an acceptance test.

Misunderstandings surface while the change is still words, not code

	Without a spec	With OpenSpec
Where intent lives	Tickets, chats and people's heads	A proposal and delta in the repo
What reviewers check	The diff, guessing at the intent	The code against an agreed spec
When gaps are found	After the code is built	While the change is still words

Changing a sentence is cheaper than changing code.

Scenarios give everyone the same definition of done

Product and stakeholders

Read the requirements in plain English and confirm they are right.

Developers and AI assistants

Build to the scenarios instead of to a guess.

Reviewers and QA

Check that every scenario has a passing test.

openspec validate checks the structure, so a requirement without a scenario is caught early.

3 · WHY IT PAYS OFF

Archiving keeps the specs current, so the documentation does not drift

`changes/add-2fa/`

The finished change: proposal, design, tasks and spec delta.

archive

`specs/auth/spec.md`

Now includes the new requirements: the system as it is today.

`changes/archive/2026-11-04-add-2fa/`

Keeps the why: the proposal and design, dated.

The specs say what the system does. The archive explains how it got there.

3 · WHY IT PAYS OFF

Plain Markdown in Git fits the tools you already use

Lives with the code

Specs are Markdown files, versioned with the code and reviewed in pull requests.

No lock-in

No server, database or new language. Any stack works, and the files stay readable without the tool.

Ready for AI

AI assistants read the specs as context and follow the same propose, apply, archive steps.

COMMON CONCERNS

It is lighter than it looks

The worry	In practice
“Is this waterfall?”	No. You spec one small change at a time, and the specs grow with the code.
“Do we spec everything first?”	No. Start with the next change; specs build up as you touch each area.
“More paperwork?”	A proposal is a page or two, and the AI assistant drafts it. You review.
“Are we locked in?”	No. It is Markdown in your repo; stop using it and nothing breaks.

NEXT STEP

Try it on one small change this sprint

1	**Set up**	Run openspec init in the repo to add the folders and commands.
2	**Pick a change**	Choose one small, real change, such as a bug fix or minor feature.
3	**Propose and review**	Run /opsx:propose, then read the proposal together.
4	**Build and archive**	Run /opsx:apply, then /opsx:archive.
5	**Look back**	Ask what the spec caught, and keep what helped.

IN SUMMARY

Agree on the change in writing, then build it

Simple

Two folders, three steps.

Safer

Gaps found in words, not code.

Compounding

Living specs, in Git, for people and AI.